

Presentation Speaker Script

Estimated total time: ~25 minutes (excluding live demo)

Slide 1 -- Title

Welcome everyone. Today I want to present a proof of concept we built to evaluate Quarkus as an alternative to Spring Boot for our Java back-end services, specifically for EU Login integration via OpenID Connect.

Slide 2 -- Context

As you know, our Java applications typically run on Spring Boot 3.x, deployed either as standalone JARs or as WARs inside Tomcat. EU Login is our mandatory authentication provider, and we've been using it through CAS and more recently through OpenID Connect.

The question we wanted to answer is simple: if we were starting a new service today, should Spring Boot be the only option on the table? To find out, we built the same EU Login OIDC integration in both frameworks and compared them head to head.

Slide 3 -- What is Quarkus?

So what is Quarkus? It's a Java framework created by Red Hat, first released in 2019, and designed from the ground up for cloud-native and container-first workloads.

The key innovation is build-time processing. Traditional Java frameworks like Spring do a lot of work at startup: classpath scanning, annotation processing, proxy generation, dependency injection wiring. Quarkus moves all of that to compile time. The result is that when your application starts, most of the heavy lifting is already done. This is what gives it dramatically faster startup times and a smaller memory footprint.

It's built on proven, vendor-neutral standards. Jakarta REST -- formerly JAX-RS -- is the standard API for building RESTful web services, the same one used in Java EE and Jakarta EE for years. CDI, Contexts and Dependency Injection, is the standard dependency injection model for Jakarta EE -- similar in purpose to Spring's DI but standardized across vendors. MicroProfile is a set of specifications for building microservices in Java -- things like health checks, metrics, fault tolerance, OpenAPI, and config. And Vert.x is the reactive toolkit that powers Quarkus' HTTP layer, giving it non-blocking I/O without requiring you to write reactive code.

The other major feature is **GraalVM native compilation**. You can compile your Quarkus application into a standalone native executable -- no JVM needed at runtime. Startup drops to tens of milliseconds and memory to tens of megabytes. This is particularly relevant for serverless workloads and scale-to-zero scenarios.

Quarkus is backed by Red Hat, it's part of their enterprise middleware portfolio, and it's used in production by organizations at scale. It's not an experiment -- it's a mature, supported platform.

Slide 4 -- Side-by-Side: EU Login OIDC Integration

Now let's look at the actual comparison. This table shows the same EU Login OIDC integration implemented in both frameworks.

Starting with OIDC configuration: in Spring Boot, we needed the `application.properties` file plus three separate Java configuration classes -- about 120 lines of code. In Quarkus, the entire OIDC setup lives in `application.properties`. Zero configuration classes.

For PKCE -- which EU Login mandates -- Spring Boot requires you to manually create an authorization request resolver, call the `withPkce()` customizer, and wire it into the security filter chain. In Quarkus, it's a single property: `pkce-required` equals `true`.

Token signature verification is another interesting difference. EU Login signs its ID tokens with PS512. Spring Boot defaults to RS256, so you need a custom `JwtDecoderFactory` bean to override the algorithm. Quarkus auto-discovers the signing algorithm from the OIDC provider's metadata endpoint. No code needed.

Now, client authentication is worth discussing in more detail. Both frameworks support the secure authentication methods that EU Login requires -- neither sends the client secret in plain text. The Spring Boot demo uses `private_key_jwt`: the application holds an elliptic curve keypair, signs a JWT with the private key, and EU Login verifies it with the public key. This is the most secure option because the secret material never leaves the client. However, it requires generating and managing a keypair, storing it in configuration, and writing custom code to wire up the Nimbus JWK source, the JWT resolver, and the token response client -- that's what those three config classes are for.

The Quarkus demo uses `client_secret_jwt`: the client and EU Login share a symmetric secret, and the client signs a JWT with it. It's simpler -- one property instead of three classes -- and still secure because the secret itself is never transmitted. Quarkus handles the JWT creation and signing internally. The trade-off is that both sides know the secret, whereas with `private_key_jwt` only the client holds the private key.

To be clear, Quarkus also supports `private_key_jwt` if you need it. The point here is that for a straightforward integration, Quarkus gets you there with significantly less ceremony.

For security rules, Spring Boot uses a `SecurityFilterChain` bean with a fluent API. Quarkus uses declarative permission rules in properties -- you define which paths require authentication and which roles, and the framework enforces them.

And finally, for local development: the Spring Boot demo requires a connection to a real EU Login server. Our Quarkus setup uses Keycloak DevServices, which automatically starts a Keycloak container in Docker when you run in dev mode. No registration, no network dependency, no setup. This was actually the reason we built this POC -- we couldn't get an EU Login client registration, and Quarkus let us develop the full OIDC flow anyway.

Slide 5 -- Code Volume Comparison

Here's the concrete file breakdown. The Spring Boot project needs SecurityConfiguration for the filter chain and PKCE, OAuth2ClientConfig for the JWK source and token client, JWTConfig for the ID token decoder override, a controller, and two Thymeleaf templates.

The Quarkus project has application.properties for the entire OIDC and security configuration, and then only business logic: the protected resource endpoints, the user resource, and the session context. Zero boilerplate configuration classes. Everything that Spring Boot needs custom Java code for, Quarkus handles through convention and properties.

Slide 6 -- Developer Experience

Developer experience is where Quarkus really shines in day-to-day work.

Live reload in Spring Boot means Spring DevTools restarts the application context. In Quarkus, changes are picked up without a restart -- you save the file, hit the endpoint, and the change is live. It also runs your tests continuously in the background.

For local OIDC testing, as I mentioned, Spring Boot needs a real server. Quarkus DevServices starts a Keycloak container automatically -- you just need Docker running. It imports a realm file with test users and roles, and the Dev UI at /q/dev-ui gives you tools to acquire and inspect tokens interactively.

For testing, Spring Boot has @WithMockUser. Quarkus has @TestSecurity for unit tests and KeycloakTestClient for integration tests against the DevServices Keycloak.

Slide 7 -- Deployment Models

Deployment is an important topic for us. Spring Boot gives us two options: standalone JAR with embedded Tomcat, or WAR deployed to an external Tomcat.

Quarkus gives us three: standalone JAR with embedded Vert.x, uber-JAR as a single fat JAR, and native executable compiled with GraalVM. There's also built-in support for building container images with Jib or

Docker.

The important thing to note: Quarkus does not deploy to Tomcat. It embeds its own HTTP server based on Vert.x and Netty. This is a deliberate design choice, not a limitation, and the next slide explains why.

Slide 8 -- Why Not Tomcat?

Tomcat deployment adds operational complexity that we've all dealt with: managing Tomcat versions and patches separately from the application, dealing with shared classloader issues when multiple WARs are deployed together, splitting configuration between the application and the container, and scaling the entire Tomcat instance when only one application needs more capacity.

Quarkus' standalone model avoids all of this. One artifact, one process. The same artifact runs in dev, test, and production -- it's immutable. The small image size and fast startup make it ideal for container and Kubernetes deployments. This aligns with where our infrastructure is heading.

Slide 9 -- Performance Characteristics

These are indicative numbers for a typical OIDC web application like ours.

On the JVM, Spring Boot starts in roughly 2 to 4 seconds and uses 150 to 250 megabytes of resident memory. Quarkus on the JVM starts in under 1.5 seconds and uses 80 to 130 megabytes. That's already a meaningful improvement.

But the native compilation numbers are where it gets dramatic: 20 to 50 milliseconds startup and 30 to 50 megabytes of memory. That's fast enough for serverless functions and scale-to-zero scenarios where cold start latency matters.

For long-running services where peak throughput is the priority, JVM mode is still recommended -- the JIT compiler optimizes hot paths over time. Native is best for startup-sensitive workloads.

Slide 10 -- Migration Path

For teams that are already comfortable with Spring, Quarkus offers compatibility extensions. You can use `@Autowired`, `@Component`, `@RestController`, `@GetMapping`, `@PreAuthorize`, and Spring Data repositories -- all compiled at build time by Quarkus.

This means a team can start with familiar Spring annotations and gradually adopt CDI and Jakarta REST as they get comfortable. It lowers the barrier to entry significantly.

One important caveat: this is a compatibility layer, not a full Spring runtime. It covers the most common patterns but not the entire Spring ecosystem.

Slide 11 -- When to Choose Quarkus

Quarkus is a good fit for new microservices and APIs, container and Kubernetes deployments, serverless functions, and any application where startup time and memory footprint matter. It's also a good choice for teams that are open to adopting Jakarta EE and MicroProfile standards.

Where it's less ideal: if you have a large existing Spring Boot codebase, the migration cost may not be justified. If you must deploy as a WAR to existing Tomcat infrastructure due to organizational constraints, Quarkus won't fit that model.

And if your application relies heavily on Spring ecosystem libraries that don't have Quarkus equivalents, you'll hit gaps. Some concrete examples: Spring Batch for batch processing -- Quarkus has no direct equivalent, though you could use JBeret via the Quarkiverse. Spring Integration and Spring Cloud Stream for enterprise integration patterns and message-driven microservices -- Quarkus offers SmallRye Reactive Messaging, but the programming model is quite different. Spring Cloud Config and Spring Cloud Gateway -- Quarkus has its own config system and no built-in API gateway. Spring HATEOAS for hypermedia-driven REST APIs has no Quarkus counterpart. And Spring Statemachine for state machine abstractions -- there's nothing equivalent in the Quarkus ecosystem.

That said, for the core use cases we have -- REST APIs with OIDC authentication, database access, and JSON processing -- Quarkus covers everything we need.

Slide 12 -- Live Demo

Now let me show you this in action.

[Start the application with `./mvnw quarkus:dev`]

I'm starting the application in dev mode. Notice in the logs that Quarkus automatically started a Keycloak container -- that's DevServices at work.

[Open `http://localhost:8080`]

This is our demo dashboard. It shows the six steps of the OAuth 2.0 Authorization Code flow. Right now we're at step 1 -- the user has visited the app but is not authenticated.

[Click "Login with EU Login"]

I'm clicking login, and you can see the browser redirects to the Keycloak login page. In production, this would be the EU Login page. Let me log in as test1 with password test -- this user has the administrator role.

[After login]

And we're back. All six flow steps are now green -- the authorization code was exchanged for tokens using PKCE, and a session was established. You can see the user's identity: username, full name, email, and the administrator role badge.

[Test the endpoints]

Now let's test role-based access. The /hello endpoint is public -- 200 OK. The /rest/me endpoint returns my user info --

200. The /api/protected endpoint requires the administrator role -- 200, I have that role. But /api/protected/editor requires the editor role -- 403 Forbidden. The server correctly denied access.

[Logout and login as editor1]

Let me logout and login as editor1. Now I have the editor role. If I test the same endpoints -- the editor endpoint returns 200, but the admin endpoint returns 403. The roles are enforced correctly.

[Show Dev UI]

Finally, let me show you the Dev UI at /q/dev-ui. Here you can see the OIDC provider configuration, acquire tokens, and inspect their contents. This is invaluable during development.

Slide 13 -- Summary

To wrap up: both frameworks successfully integrate with EU Login via OpenID Connect. Spring Boot is the industry standard and works well. But Quarkus achieves the same result with zero configuration classes, better developer experience through DevServices and live reload, faster startup, and lower memory usage.

Our recommendation is not to replace Spring Boot across the board, but to consider Quarkus as a viable option for new services, especially those targeting containerized deployments.

Slide 14 -- Questions?

I've listed the key resources here. The two demo projects are available for you to try. Happy to take any questions.